

MathAgent: A SciPy-Augmented Benchmark for Agentic Scientific Computation

Kishor Kumar Bhaumik

Shivanshu Tripathi

Maziar Raissi

Abstract

Numerical computation underpins scientific and engineering practice, where answers to questions about reactor dynamics, singular differential equations or asymptotic scaling must be derived rather than recalled. Language models that rely on parametric reasoning alone are poorly suited to such problems, since without code execution they fall back on heuristic approximations. A natural remedy is to pair the model with a Python execution environment in a ReAct-style agent loop, and a common assumption is that this access improves accuracy. We test that assumption with MATHAGENT, a SciPy-augmented benchmark of 250 applied-mathematics problems across five domains spanning a computational-to-analytical spectrum, each with ground-truth answers fixed by reference solvers. Evaluating four model families across three levels of tool access, we find the value of tool access to be conditional: it helps most when a problem requires computation the model cannot perform reliably on its own and it can reduce accuracy when the model already reasons well about a domain and the mandated code path displaces that reasoning. GPT-5.5, Qwen2.5-7B, and Qwen3.6-27B improve with tooling (by 14.0, 10.4, and 18.0 points), whereas Claude Opus 4.8 declines by 4.4 points. We trace these losses to faulty problem-to-code translation and distill a practical rule for when agentic tool use is worth deploying. The best configuration reaches 71.6%, leaving substantial headroom. Our code is available at <https://anonymous.4open.science/r/MathAgent-F70A>.

1 Introduction

Numerical computation is central to scientific and engineering practice. Whether simulating reactor dynamics, identifying finite-time singularities in differential equations, or applying asymptotic methods to reduce complex systems, practitioners routinely need precise quantitative answers that cannot

be approximated through reasoning alone. As large language models (LLMs) are increasingly deployed in scientific workflows, a natural question arises: can they reliably solve such problems?

The evidence suggests they cannot, at least not unaided. Parametric reasoning, the ability to produce answers from learned representations without executing code, is inherently imprecise for numerical tasks. A model that has absorbed textbook derivations of the Laplace method cannot reliably compute the asymptotic value of a specific integral to six decimal places without actually performing the computation. Our experiments confirm this: no-tool baselines across four model families fall between 22.8% and 57.6% accuracy on our benchmark, well below what scientific applications require. Tool-augmented agents offer a principled remedy. By pairing an LLM controller with a code execution environment, these systems delegate numerical work to deterministic solvers, combining the flexibility of language understanding with the precision of computation. Frameworks such as ReAct (Yao et al., 2022) and Toolformer (Schick et al., 2023) established the foundations for this paradigm, and recent work has demonstrated its promise in chemistry (M. Bran et al., 2024) and geoscience (Zhang et al., 2024). Applied mathematics, with its mix of numerical integration, symbolic manipulation, and asymptotic analysis, is a natural and demanding testbed for such systems.

We introduce MATHAGENT, a SciPy-augmented benchmarking system that evaluates LLMs across five applied-mathematics domains spanning a computational-to-analytical spectrum (Section 3.1). The benchmark comprises 250 problems generated from templates, with ground-truth answers computed by reference solvers. We evaluate four model families, GPT-5.5, Claude Opus 4.8, Qwen2.5-7B, and Qwen3.6-27B, across three levels of tool access that form an interface-complexity ladder: no tools, single-shot code generation, and a full itera-

tive agent loop.

A common assumption, reflected in tool-integrated reasoning systems such as ReAct (Yao et al., 2022), PAL (Gao et al., 2023), and ToRA (Gou et al., 2024), is that access to code execution improves a model’s performance on quantitative tasks. Our results show that this benefit is real but conditional. GPT-5.5, Qwen2.5-7B, and Qwen3.6-27B improve at every level of added tooling, gaining 14.0, 10.4, and 18.0 points. Claude Opus 4.8 moves in the opposite direction, finishing 4.4 points below its no-tool baseline. The effect is domain- and backbone-specific: on dominant-balance analysis, the same problems yield an 18-point gain for GPT-5.5 and a 22-point gain for Qwen3.6-27B, yet a 17-point loss for Opus 4.8. Capability alone does not explain this, since Qwen3.6-27B and Opus 4.8 begin from nearly identical no-tool baselines and diverge sharply once the agent loop is introduced (Section 4). The losses trace to faulty problem-to-code translation, where the agent writes code that runs successfully but does not faithfully represent the problem.

Our main contributions are:

- **A practical account of when agentic tool use helps.** Across four model families and three levels of tool access, we characterize the conditions under which code execution improves accuracy and the conditions under which it degrades it and distill a deployment rule for agentic numerical problem solving.
- **A solver-verified applied-mathematics benchmark.** 250 problems across five domains spanning a computational-to-analytical spectrum, generated from templates with ground-truth answers fixed by SciPy reference solvers. The best configuration we evaluate reaches 71.6% accuracy.
- **A failure-mode decomposition of agent regressions.** We show that tool-induced accuracy losses stem mainly from faulty problem-to-code translation rather than harness artifacts, and that they track a domain-by-backbone interaction rather than raw model capability.

2 Related Work

Tool-Augmented Language Models. The integration of external tools into LLM inference has produced substantial gains on reasoning-heavy

tasks. ReAct (Yao et al., 2022) established the paradigm of interleaving chain-of-thought reasoning with tool invocations, and Toolformer (Schick et al., 2023) showed that models can learn when and how to call tools from self-supervised objectives. For mathematical tasks, Program-Aided Language Models (Gao et al., 2023) and Program of Thoughts (Chen et al., 2022) showed that delegating computation to a Python interpreter substantially improves arithmetic and algebraic reasoning. Closest to our setting, ToRA (Gou et al., 2024) trains tool-integrated reasoning agents that interleave natural-language reasoning with calls to computation and symbolic-solver libraries, reporting consistent gains on competition mathematics. MATHAGENT builds on this tradition but differs in scope and aim. Rather than training models or targeting word problems, we evaluate off-the-shelf models on problems that require scientific-computing libraries, and we compare three levels of tool access on a single benchmark to isolate the effect of interface complexity and to identify the conditions under which that access stops helping.

Agents for Scientific Domains. Recent work has extended tool-augmented agents into domains requiring specialized computation. ChemCrow (M. Bran et al., 2024) augments LLMs with chemistry tools including reaction predictors and safety checkers, and Boiko et al. (2023) demonstrated closed-loop experimental planning for chemical synthesis. Applied mathematics, with its mix of numerical integration, symbolic manipulation, and asymptotic analysis, remains underexplored as a testbed for such systems. We address this gap and show that tool benefit is not guaranteed even at frontier capability: the determining factor is the fit between problem character and the model’s reasoning style, not scale alone.

Mathematical Reasoning Benchmarks. Approaches such as Minerva (Lewkowycz et al., 2022) push parametric quantitative reasoning to college-level science and engineering problems without external tools, while widely used benchmarks such as GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) focus on symbolic and algebraic problem solving, where answers follow from logical manipulation of exact expressions. Mathbench (Liu et al., 2024) further evaluates both theoretical and applied mathematical proficiency through a hierarchical benchmark. Our benchmark targets a different regime: problems whose solu-

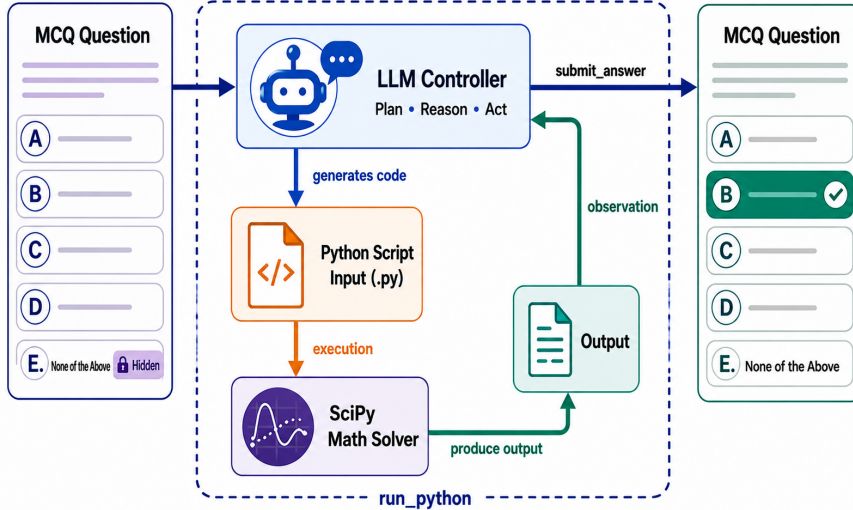


Figure 1: The MATHAGENT agent loop. Given an MCQ question (left), the LLM controller operates in a ReAct-style loop with two tools, `run_python` and `submit_answer`. Each `run_python` invocation proceeds through three phases: the controller generates a Python script, the harness executes it against SciPy, and the resulting stdout is returned to the controller as an observation. The controller repeats this cycle until it converges on an answer and calls `submit_answer` (right, correct option highlighted). Option E (*None of the Above*) is hidden from the model throughout the loop, so the model cannot fall back on a random guess that might be correct by luck rather than by computation; E is assigned externally only when the step budget is exhausted.

tions require numerical integration, asymptotic approximation, or stochastic simulation at the level of SciPy computation. This distinction matters, because as our results show, models that reason competently about a domain without tools can still fail when forced to translate that analytical intuition into executable numerical code.

3 The MATHAGENT System

3.1 Benchmark Construction

MATHAGENT comprises 250 multiple-choice problems distributed evenly across five applied-mathematics domains (50 per domain), ordered along a computational-to-analytical spectrum: stochastic reactor dynamics, finite-time ODE blow-up, Laplace asymptotic approximation, dominant-balance analysis, and nondimensionalization. Each domain targets a distinct class of numerical reasoning that cannot be reliably resolved through parametric recall alone.

Problems are generated programmatically from templates. Each template encodes a canonical problem class: stochastic reactor problems require computing the probability that a concentration-governed ODE with a Gamma-distributed measurement time satisfies a validity condition, and ODE blow-up problems require locating the finite-time singularity of a nonlinear differential equation.

Parameters are sampled within controlled ranges to ensure solvability and numerical non-triviality. Ground-truth answers are computed by reference solvers using SciPy (`solve_ivp`, `quad`) and SymPy, making every answer deterministic and independently verifiable. Each problem is presented as a four-option multiple-choice question (A–D), where all options are plausible numerical values with a controlled minimum separation from the correct answer. A fifth option, E (*None of the Above*), is included in the answer key but handled separately, as described below.

3.2 Evaluation Framework

Evaluation modes. We evaluate each model under three levels of tool access that together form an interface-complexity ladder.

No-tool access. The model receives the question and four options and produces a final answer through chain-of-thought reasoning alone, with no access to a code execution environment.

Code generation. The model writes a single Python program, which the evaluator executes; the model then observes the output and selects the closest matching option. There is no iterative loop, and the model has exactly one opportunity to write and run code.

Tool-augmented agent. The model operates

Model	Mode	Stoch	Blowup	Laplace	DomBal	Nondim	Overall
GPT-5.5	No tool	47	45	21	13	18	57.6%
	Code gen	47	26	21	30	32	62.4%
	Agent	49	39	34	31	26	71.6%
	Δ	+2	-6	+13	+18	+8	+14.0
Claude Opus 4.8	No tool	26	29	30	30	15	52.0%
	Code gen	50	22	8	21	25	50.4%
	Agent	50	19	22	13	15	47.6%
	Δ	+24	-10	-8	-17	0	-4.4
Qwen3.6-27B	No tool	43	37	23	10	15	51.2%
	Code gen	46	25	12	23	29	54.0%
	Agent	46	37	33	32	25	69.2%
	Δ	+3	0	+10	+22	+10	+18.0
Qwen2.5-7B	No tool	11	8	0	25	13	22.8%
	Code gen	9	10	4	19	31	29.2%
	Agent	15	23	12	18	15	33.2%
	Δ	+4	+15	+12	-7	+2	+10.4

Table 1: Accuracy on MATHAGENT across all twelve model–mode combinations (out of 50 per domain, 250 total). Δ rows show the change from no-tool to agent mode. **Bold** overall indicates net improvement; *italic* indicates net degradation. Domain columns are ordered along the computational-to-analytical spectrum. Models are ordered by no-tool accuracy descending.

within a ReAct-style loop (Yao et al., 2022) with access to two tools: `run_python`, which executes arbitrary Python code using `numpy`, `scipy`, and `sympy` and returns `stdout`, and `submit_answer`, which records a final answer choice and terminates the loop. The agent runs for up to $T_{\max} = 10$ steps.

Agent system prompt. The system prompt is structured around three blocks. The *role block* defines the agent as an applied mathematics expert whose task is to solve problems by writing and executing code rather than reasoning from memory. The *tool block* specifies the interface contract for each tool: `run_python` requires a code snippet that prints its final result as the last line of output, and `submit_answer` requires a single letter from A to D. The *rules block* enforces tool-first behavior: the agent must call `run_python` for any numerical computation, may issue only one tool call per turn, and is instructed that if the computed result does not match any listed option, the code must be revised and re-run. This last rule promotes iterative self-correction and discourages early option-matching on approximate results.

The hidden-E mechanism. Option E (*None of the Above*) is never presented to the model during evaluation: all three modes show only options A through D. Without this design, a model facing a difficult or uncertain problem could select E at any point, obtaining a potentially correct outcome without performing any computation, which

would confound the measurement of tool benefit. Hiding E forces the model to engage numerically and reserves it as an externally assigned marker of failure for a trajectory that exhausted the step budget or produced no parseable answer. Since the correct answer is always one of A–D by construction, E is never the right choice, and its assignment rate measures how often an agent fails to converge within the step budget. This rate varies across backbones: step-budget exhaustion occurs on 5.2% of problems for GPT-5.5, 7.6% for Claude Opus 4.8, 5.6% for Qwen3.6-27B, and 26% for Qwen2.5-7B, reflecting markedly different loop efficiency and informing the failure analysis in Section 5.

Models. We evaluate four model families: GPT-5.5 and Claude Opus 4.8 through their respective inference APIs, and Qwen2.5-7B-Instruct (Yang et al., 2025) and Qwen3.6-27B run locally in 16-bit precision (bfloat16) with the transformers library. All three evaluation modes are applied to each model, giving twelve model–mode combinations in total.

4 Results

4.1 Overall Performance

Table 1 reports accuracy across all twelve model–mode combinations. The highest overall accuracy is GPT-5.5 with the full agent loop (71.6%), and the lowest is Qwen2.5-7B without tools (22.8%). The best configuration reaches 71.6%, indicating

Model	Step-limit	Exec error	Wrong interp.
GPT-5.5	13	7	51
Claude Opus 4.8	19	5	107
Qwen3.6-27B	14	8	55
Qwen2.5-7B	65	58	44

Table 2: Agent error decomposition by failure category. *Wrong interp.* denotes trajectories where code executed without error but the submitted answer was incorrect.

that the benchmark remains challenging across the full range of model capability.

4.2 The Interface-Complexity Ladder

Figure 2 plots accuracy along the interface-complexity ladder for all four models. GPT-5.5, Qwen2.5-7B, and Qwen3.6-27B improve at every rung, with net gains of +14.0, +10.4, and +18.0 points. Claude Opus 4.8 moves the other way at each step, from 52.0% to 50.4% to 47.6%, a net loss of 4.4 points. To our knowledge, this is the first report of a frontier model declining monotonically under its own agent loop. We read it as an existence proof rather than a property of frontier scale, since GPT-5.5 is also a frontier model and improves throughout.

The reversal is not uniform across domains. Opus 4.8 gains substantially on stochastic reactor problems under tool augmentation (+24 from no-tool to agent), a domain that requires a six-decimal-place probability from ODE integration and Gamma quadrature, where parametric recall cannot reach the required precision. On the other domains it stays flat or regresses. The contrast is sharpest on dominant balance, where tools give GPT-5.5 an 18-point gain and Qwen3.6-27B a 22-point gain but cost Opus 4.8 17 points. Notably, Qwen3.6-27B and Opus 4.8 enter the agent loop within 0.8 points of each other (51.2% vs. 52.0%) yet finish 21.6 points apart, which indicates that raw capability is not what determines tool benefit.

5 Analysis

5.1 Failure Mode Decomposition

Table 2 decomposes agent errors into three categories: *step-limit exhaustion*, where the agent exceeds $T_{\max} = 10$ steps and is assigned option E; *execution error*, where every `run_python` call raises an exception; and *wrong interpretation*, where at least one execution succeeds but the agent submits an incorrect answer.

Wrong interpretation is the dominant failure mode

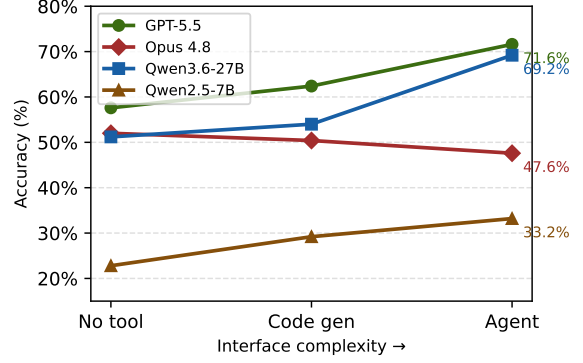


Figure 2: Accuracy along the interface-complexity ladder for all four model families. GPT-5.5, Qwen3.6-27B, and Qwen2.5-7B improve with added tooling, while Claude Opus 4.8 declines at each step, ending 4.4 points below its no-tool baseline.

for three of the four backbones. GPT-5.5 and Qwen3.6-27B attribute 72% of their errors to this category (51 of 71 and 55 of 77 respectively), and Claude Opus 4.8 attributes 82% (107 of 131). For all three, step-limit exhaustion and execution errors together account for fewer than 20 failures each, which confirms that the primary bottleneck is not the ability to invoke or run tools but faulty problem-to-code translation: the agent produces code that executes without error yet does not faithfully represent the problem.

Qwen2.5-7B shows a different profile. Step-limit exhaustion (65) and execution errors (58) together account for 73% of its 167 failures, indicating that the model frequently cannot generate valid SciPy calls within the step budget. Its wrong-interpretation count (44) is the lowest of the four models in absolute terms, not because it reasons more accurately but because it rarely produces working code to misinterpret in the first place.

The decomposition also explains the Qwen3.6-27B versus Opus 4.8 divergence from Section 4.2. Both models have low step-limit and execution-error counts, so both can reliably write and run code. The gap lies in wrong-interpretation failures: Opus 4.8 accumulates 107 such errors against 55 for Qwen3.6-27B, despite a nearly identical no-tool baseline. Those 52 additional wrong-interpretation failures, on a matched capability baseline, account for the 21.6-point divergence in agent accuracy and localize the regression to problem-to-code translation on analytical domains rather than to any general inability to use tools.

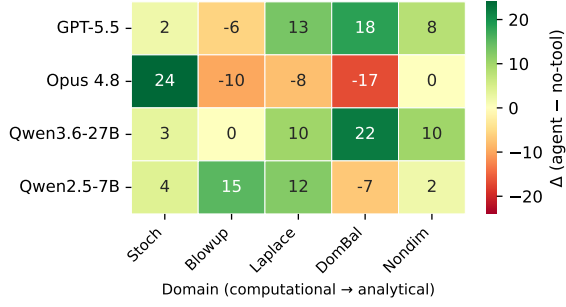


Figure 3: Per-domain Δ (agent – no-tool) for each backbone, ordered along the computational-to-analytical spectrum. The stochastic reactor column is uniformly green: every model benefits from tool access on this domain, since the correct answer requires numerical integration that parametric reasoning cannot reliably approximate. The dominant-balance column shows the largest spread, with +18 for GPT-5.5, +22 for Qwen3.6-27B, and -17 for Claude Opus 4.8 on identical questions, pointing to a backbone-specific interaction between analytical reasoning style and the forced code path.

5.2 Domain-Dependence of Tool Benefit

Figure 3 shows that tool benefit depends jointly on the domain and the backbone. Stochastic reactor is the only domain where every model gains under tool augmentation (+2, +24, +3, +4 for GPT-5.5, Opus 4.8, Qwen3.6-27B, and Qwen2.5-7B), consistent with its purely computational character: the correct answer requires a numerical integral that no model can approximate reliably from memory. On the analytical domains the picture is more varied. On dominant balance, GPT-5.5 and Qwen3.6-27B gain 18 and 22 points while Opus 4.8 loses 17 on identical questions, which rules out the possibility that the domain is inherently unsuited to agents.

A consistent pattern explains much of this variation: tool benefit tends to decrease as a model’s unaided competence on a domain increases. When a model already answers a domain well without tools, mandating a code path can replace reliable parametric reasoning with a plausible but incorrect computation; when it answers poorly, the same code path supplies grounding the model otherwise lacks. Opus 4.8 enters the loop with strong no-tool performance on dominant balance (30/50) and Laplace (30/50) and loses ground on both, whereas GPT-5.5 enters the same domains with weaker no-tool performance (13/50 and 21/50) and gains. The stochastic reactor is the natural boundary case: computation is unavoidable there, so every model benefits regardless of its parametric baseline. The

same logic accounts for the Qwen3.6-27B versus Opus 4.8 divergence reported in Section 4.2. From near-identical baselines, the model whose analytical reasoning is more readily displaced by the forced code path ends well behind. Taken as a practical rule, tool access is most valuable when a problem cannot be solved without computation, and least valuable, sometimes harmful, when the model already reasons well and the code path replaces a reliable heuristic with a fragile one.

6 Discussion

When should tool augmentation be deployed?

Our results suggest a practical heuristic: tool augmentation is beneficial only when the underlying model has sufficient capability to (i) select the correct tool, (ii) map question parameters to tool inputs, and (iii) interpret numerical results against the available options. Below this threshold, the cost of agent scaffolding (step-limit exhaustion, tool misuse, and context-management failures) exceeds the benefit of access to deterministic solvers. The near-identical no-tool baselines of Qwen3.6-27B and Claude Opus 4.8, paired with their divergent agent trajectories, further suggest that the threshold is not simply a function of scale: reasoning style and domain fit play an independent role.

Implications for benchmark design. MATHAGENT shows that benchmark design choices can isolate distinct agent skills. By providing structured parameters, we separate the *mathematical reasoning* component from the *parameter extraction* component, which enables finer-grained diagnosis of failure modes.

7 Conclusion

We presented MATHAGENT, a solver-verified benchmark of 250 applied-mathematics problems and used it to study when agentic tool use helps off-the-shelf language models on numerical problems. Tool access is most valuable when a problem cannot be solved without computation and least valuable, sometimes harmful, when a model already reasons well about a domain and the mandated code path displaces that reasoning. The dominant failure is not tool misuse but faithful-looking code that solves the wrong problem. For practitioners, this argues for enabling agentic execution selectively, conditioned on whether the underlying model can map a problem to code reliably for the domain at hand, rather than by default.

Limitations

Our benchmark contains 250 questions, a size dictated by the cost of API inference and the time required for local model evaluation. A larger question set would tighten confidence intervals and allow finer-grained per-domain conclusions. All solvers are built on SciPy and SymPy, so we cannot claim that the observed capability threshold generalizes to other scientific computing libraries such as torch.optim or domain-specific simulators without replication. The four-choice MCQ format provides a 25% random-guess floor and may inflate accuracy relative to free-form numerical answering. Finally, we do not report a human expert baseline; all gains are measured relative to LLM no-tool performance and compute constraints limit the number of model families we evaluate, so the exact location of the capability threshold may shift as new models are released (e.g. Claude Fable 5).

Ethics Statement

MATHAGENT consists of applied mathematics problems and does not contain personal, sensitive, or copyrighted data. The benchmark is intended for evaluating LLM-based tool use in scientific computing contexts and carries no foreseeable misuse beyond standard LLM evaluation risks. Models used for evaluation are accessed through their standard APIs or open weights and used in accordance with their respective terms of service.

References

- Daniil A Boiko, Robert MacKnight, and Gabe Gomes. 2023. Emergent autonomous scientific research capabilities of large language models. *arXiv preprint arXiv:2304.05332*.
- Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. In *International conference on machine learning*, pages 10764–10799. PMLR.
- Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujia Yang, Minlie Huang, Nan Duan, Weizhu Chen, and 1 others. 2024. Tora: A tool-integrated reasoning agent for mathematical problem solving. In *International Conference on Learning Representations*, volume 2024, pages 48362–48395.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, and 1 others. 2022. Solving quantitative reasoning problems with language models. *Advances in neural information processing systems*, 35:3843–3857.
- Hongwei Liu, Zilong Zheng, Yuxuan Qiao, Haodong Duan, Zhiwei Fei, Fengzhe Zhou, Wenwei Zhang, Songyang Zhang, Dahua Lin, and Kai Chen. 2024. Mathbench: Evaluating the theory and application proficiency of llms with a hierarchical mathematics benchmark. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 6884–6915.
- Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. 2024. Augmenting large language models with chemistry tools. *Nature machine intelligence*, 6(5):525–535.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Hao Zhang, Jin-Jian Xu, Hong-Wei Cui, Lin Li, Yaowen Yang, Chao-Sheng Tang, and Niklas Boers. 2024. When geoscience meets foundation models: Toward a general geoscience artificial intelligence system. *IEEE geoscience and remote sensing magazine*.

Appendices

A System Prompt

The following prompt is passed as the system message to the LLM controller at the start of every agent trajectory, identical across all backbones and all questions. No question-specific content appears in the system prompt; the MCQ question and options are passed separately as the first user message. The tool block embedded below is reproduced exactly in Appendix B.

```
You are an applied mathematics expert.
Solve the given problem by writing and
executing Python code.
Available libraries: numpy (np), scipy,
sympy (sp).
```

You have access to two tools:

- run_python**
Write Python code using numpy, scipy, sympy. Your code MUST print() the final answer as the last line.
Format:
ACTION: run_python
```python  
# your code here  
print(answer)  
```
- submit_answer**
When you are confident, submit your final answer.
Format:
ACTION: submit_answer
CHOICE: <A or B or C or D>

Rules:

- You MUST use run_python for any numerical computation. Do NOT answer from memory.
- Call only ONE tool per turn.
- After you see the output, decide: is this the correct answer? If yes, submit_answer.
- The answer is definitely one of A, B, C, or D.
- If your result does not match any option, your code is wrong. Fix it and try again.

B Tool Schemas

MATHAGENT does not use structured function-calling APIs. Instead, tools are invoked through a plain-text ReAct protocol that is parsed directly from the model’s output, making the interface identical across all backends (Anthropic, OpenAI and local HuggingFace models) without any provider-specific configuration. The tool block given to the agent as part of its system prompt is the one shown in Appendix A.

Parsing. At each step the harness scans the model’s raw text output for ACTION: run_python or ACTION: submit_answer. For run_python, the code block is extracted between the opening “python and closing “” delimiters and passed to the execution backend. For submit_answer, the

CHOICE: line is read and validated against {A, B, C, D}. Any response matching neither pattern is classified as unknown: the model receives a format reminder and the step counts against the $T_{\max}$ budget without producing a tool observation.

Execution and observation. Extracted code is run in a subprocess with a 30-second wall-clock timeout. The observation fed back to the model is one of four possible strings: (i) stdout on successful execution, optionally followed by solver warnings from stderr (e.g. lsoda stiffness notices, truncated to 500 characters); (ii) the last 15 lines of stderr on exception; (iii) a timeout message instructing the model to rewrite with a finite stopping condition; or (iv) a reminder to add a print() call if stdout is empty.

C Example Trajectories

We present two representative agent trajectories from GPT-5.5: one successful and one illustrating the dominant wrong-interpretation failure mode. Code is lightly trimmed for space; full trajectories are available in the released result files.

Successful trajectory — stochastic reactor (STOCH_01)

Steps: 3 Executions: 2 Result: Correct (option C) ✓

Question: A chemical reactor’s concentration $C(t)$ evolves under $\dot{C} = -k(t)C + S(t)$, $C(0) = 2.3986$, with $k(t) = 0.7449/(1 + 0.0849t)$ and $S(t) = 4.3835 e^{-0.1806t} \sin(0.4043t)$. A sensor fires at $T \sim \text{Gamma}(\alpha = 2.1192, \beta = 0.6405)$ (rate). A reading is valid iff $T \in [2.92, 6.0]$ and $C(T) > 1.1591$. Compute $P = \Pr(\text{valid})$ to six decimal places.

Step 1. The model writes an ODE solver combined with Gamma CDF integration but uses the deprecated `np.math.gamma()`, triggering an `AttributeError`:

```
OBSERVATION:
Error: AttributeError: module 'numpy'
has no attribute 'math'.
Did you mean: 'emath'?
```

Step 2. The model rewrites the code, replacing `np.math.gamma` with `scipy.stats.gamma.cdf`:

```
from scipy.integrate import solve_ivp
from scipy.optimize import brentq
```

```
from scipy.stats import gamma

sol = solve_ivp(rhs, (0, hi), [C0],
               rtol=1e-12, atol=1e-14,
               dense_output=True, max_step=0.001)

prob = sum(
    gamma.cdf(y, a=alpha, scale=1/beta) -
    gamma.cdf(x, a=alpha, scale=1/beta)
    for x, y in intervals)
print(f"{prob:.6f}")
```

OBSERVATION:
stdout: 0.359219

Step 3. The model matches 0.359219 to option C and submits correctly.

Note. This trajectory illustrates the agent’s self-correction capability: the step-1 execution error was recovered in a single retry. The problem cannot be solved without tools, since the correct value requires integrating a nontrivial ODE over a Gamma-distributed window, which no model can reliably recall from training data.

Wrong-interpretation failure — dominant balance (DOMBAL_108)

Steps: 2 **Executions:** 1 **Result:** Incorrect (submitted D, expected C) ✗

Question: Consider $I(\epsilon) = \int_0^{24.80} \frac{1}{\epsilon + 9.6x^2 + 8.9x^8} dx$. In the large- ϵ regime, using dominant balance retaining only the lowest-degree term of $P(x)$, what is the numerical value of the analytical approximation at $\epsilon = 6.005$ (rounded to 2 d.p.)?

Options: A. 0.05 ✗ B. 0.08 ✗ C. 0.13 ✓ D. 0.21 ✗

Step 1. The model correctly identifies that the x^8 term should be dropped in the large- ϵ regime, leaving $\epsilon + 9.6x^2$ in the denominator. However, rather than deriving the closed-form dominant-balance approximation, it evaluates the simplified integrand numerically:

```
from scipy.integrate import quad
eps = 6.005
val = quad(lambda x: 1/(eps + 9.6*x**2),
           0, 24.80)[0]
print(val)
```

OBSERVATION:
stdout: 0.2026855592494646

Step 2. The model matches 0.2027 to the nearest option D (0.21) and submits. No error was raised at any point. The harness had no mechanism to detect the failure; only the ground-truth comparison revealed it.

Error Analysis: The correct answer (C, 0.13) requires applying the dominant-balance formula analytically: deriving the closed-form expression for $\int_0^L 1/(\epsilon + 9.6x^2) dx$ and evaluating it symbolically before substituting $\epsilon = 6.005$. The model bypassed the asymptotic reasoning entirely, substituted the numerical value of ϵ first and integrated numerically. The code was syntactically correct, executed cleanly and produced a plausible number, yet solved the wrong problem. This is the defining characteristic of the wrong-interpretation failure mode, which accounts for 51 of GPT-5.5’s 71 agent errors and 107 of Claude Opus 4.8’s 131.

D Distractor Sampling Details

Each question has three distractors generated programmatically by the domain’s solver, giving four options (A–D) total. All domains share the same core sampling strategy. Each distractor is a multiplicative perturbation of the correct answer, sampled as $c_i = \text{correct} \times (1 + d_i \cdot f_i)$, where $f_i \sim \text{Uniform}(0.3, 0.7)$ and $d_i \in \{-1, +1\}$ are drawn independently for each distractor. This ensures every wrong option is between 30% and 70% away from the correct value in relative terms, producing plausible alternatives that cannot be rejected by order-of-magnitude reasoning alone. A minimum absolute separation `min_gap` is enforced between every pair of options (including the correct answer); if a candidate violates this constraint it is resampled, up to 2,000 attempts. The four values are then shuffled uniformly into labels A–D.

Domain-specific parameters account for differences in value range, sign, and required precision, as summarized in Table 3.

Domain	min_gap	Rounding	Range / sign constraint
Stochastic reactor	0.005	6 d.p.	Clipped to $(10^{-6}, 1 - 10^{-6})$
ODE blow-up	0.001	6 d.p.	Sign-preserving
Dominant balance	0.03	2 d.p.	Clipped to $[0.01, \infty)$
Laplace method	0.03	2 d.p.	Sign-preserving
Nondimensionalization	0.03	2 d.p.	$ \cdot $, clipped to $[0.01, \infty)$

Table 3: Per-domain distractor sampling parameters.

Stochastic reactor and ODE blow-up use six decimal places because their correct answers are prob-

abilities and precise blow-up times respectively, where small differences between options are numerically meaningful. The three analytical domains (dominant balance, Laplace, nondimensionalization) round to two decimal places to match the precision at which asymptotic approximations are conventionally reported. Sign-preserving sampling (ODE blow-up, Laplace method) prevents a negative correct answer from producing a positive distractor, which would be trivially identifiable. Stochastic reactor values are clipped to the open interval (0, 1) since they represent probabilities; nondimensional numbers are clipped to a positive minimum of 0.01 since they represent physical scale ratios.

E Hyperparameters

API models. GPT-5.5 is called with `max_completion_tokens=4096` and `reasoning_effort=medium`; Claude Opus 4.8 is called with `max_tokens=4096`. Temperature is left at the provider default for both models. No-tool evaluations use the Anthropic and OpenAI Batch APIs, with batches completing in approximately one hour. Agent and code-generation evaluations use real-time API calls with no explicit retry logic beyond the agent’s own step-budget. API inference for GPT-5.5 cost approximately \$50 and for Claude Opus 4.8 approximately \$40, covering all three evaluation modes (no-tool batch, code generation, and agent). No-tool batch evaluations were submitted via the Anthropic and OpenAI Batch APIs at reduced cost (approximately 50% of real-time pricing). Local inference for the two Qwen models incurred no direct monetary cost beyond electricity and hardware access.

Agent loop. The step limit is $T_{\max} = 10$ across all models and modes. Each `run_python` call is executed in a subprocess with a 30-second wall-clock timeout. No other per-question time limit is imposed.

Local models. Qwen2.5-7B-Instruct and Qwen3.6-27B are loaded from HuggingFace in full 16-bit precision (`torch_dtype=torch.bfloat16`) using the transformers library; no quantization is applied. Both models run on a single NVIDIA H100 NVL (94 GB). Qwen2.5-7B-Instruct requires approximately 30 GB of GPU memory, while Qwen3.6-27B requires approximately 82 GB. Due to its size, the full Qwen3.6-27B agent evaluation

across all 250 questions required approximately six days of continuous inference.

F Sample Questions

To illustrate the nature of the benchmark, we present one representative question from each of the five domains, shown exactly as presented to the model (options A–D, with E hidden). The correct option is marked with ✓.

Stochastic reactor . A chemical reactor’s concentration $C(t)$ evolves under $\dot{C} = -k(t)C + S(t)$, $C(0) = 2.3986$, with $k(t) = 0.7449/(1+0.0849t)$ and $S(t) = 4.3835 e^{-0.1806t} \sin(0.4043t)$. A sensor fires at $T \sim \text{Gamma}(\alpha = 2.1192, \beta = 0.6405)$ (rate). A reading is valid iff $T \in [2.92, 6.0]$ and $C(T) > 1.1591$. Compute $P = \Pr(\text{valid})$ to six decimal places.

- A. 0.498659 B. 0.569868 C. 0.359219 ✓
D. 0.247640

ODE blow-up . Consider the fourth-order ODE $y'''' = 0.1225 u_0^2 + 0.3475 u_1^2 + 0.3009 u_2^2 + 0.7628 u_3^3$, with initial conditions $y(0) = 2.1917$, $y'(0) = 2.7304$, $y''(0) = 0.7173$, $y'''(0) = 1.5548$. The solution $y'(x)$ blows up at a finite point x^* . Find x^* , rounded to six decimal places.

- A. 0.267134 B. 0.305280 C. 0.192435 ✓
D. 0.088365

Laplace method. Consider the integral

$$I(x) = \int_{-1.1}^{-0.3} (-0.6t^6 + 0.4t^3 + 1.9 \sin t) \times e^{-x(-0.3t^5 - 0.9t^3 + 1.4 \cos t)} dt.$$

In the limit $x \rightarrow \infty$, the leading-order asymptotic form is $I(x) \approx A e^{cx}/x$. What is the numerical value of the pre-exponential factor A (rounded to two decimals)?

- A. -1.48 B. -0.94 ✓ C. -0.37 D. -0.55

Dominant balance . Consider the integral $I(\epsilon) = \int_0^{24.80} \frac{1}{\epsilon + 9.6x^2 + 8.9x^8} dx$. In the large- ϵ regime, using dominant balance retaining only the lowest-degree term of $P(x)$, what is the numerical value of the analytical approximation at $\epsilon = 6.005$ (rounded to two decimals)?

- A. 0.05 B. 0.08 C. 0.13 ✓ D. 0.21

Nondimensionalization . Nondimensionalize the polynomial $P(x) = 8x^{10} - 7x^2 - 8$ into the

form $\varepsilon y^{10} \pm y^k \pm 1$. Solve for ε , rounded to two decimal places.

A. 1.95 ✓ B. 0.90 C. -1.19 D. 0.84